

CHAPTER 4

STANDARD TEMPLATE LIBRARY (STL) I

IN THIS CHAPTER, YOU WILL:

- Learn about the Standard Template Library (STL)
- Become familiar with the three basic components of the STL—containers, iterators, and algorithms
- Explore how `vector` and `deque` containers are used to manipulate data in a program
- Discover the use of iterators

Chapter 2 introduced and examined templates. With the help of class templates, we developed (and used) a generic code to process lists. For example, in Chapter 3, we used the **class** `arrayListType` to process a list of integers and a list of strings. In Chapters 5, 7, and 8, we will study the three most important data structures: linked lists, stacks, and queues. In Chapter 5, using class templates, we will develop a generic code to process linked lists. In addition, using the second principle, inheritance, of object-oriented programming (OOP), we will develop a generic code to process ordered lists. Then, in Chapters 7 and 8, we will use class templates to develop a generic code to implement stacks and queues. Along the way, you will see that a template is a powerful tool that promotes code reuse.

C++ is equipped with a Standard Template Library (STL). Among other things, the STL provides class templates to process lists (contiguous or linked), stacks, and queues. This chapter discusses some of the STL's important features, and shows how to use certain tools provided by the STL in a program. Chapter 13 describes the features of the STL not described in this chapter.

In the ensuing chapters, you will learn how to develop your own code to implement and manipulate data, as well as how to use professionally written code.

Components of the STL

The main objective of a program is to manipulate data and generate results. Achieving this goal requires the ability to store data into computer memory, access a particular piece of data, and write algorithms to manipulate the data.

For example, if all the data items are of the same type and we have some idea of the number of data items, we could use an array to store the data. We can then use an index to access a particular component of the array. Using a loop and the array index, we can step through the elements of the array. Algorithms, such as those for initializing the array, sorting, and searching, are used to manipulate the data stored in an array. On the other hand, if we do not want to worry about the size of the data, we can use a linked list, as is described in Chapter 5, to process it. If the data needs to be processed in a Last In First Out (LIFO) manner, we can use a stack (Chapter 7). Similarly, if the data needs to be processed in a First In First Out (FIFO) manner, we can use a queue (Chapter 8).

The STL is equipped with these features to effectively manipulate data. More formally, the STL has three main components:

- Containers
- Iterators
- Algorithms

Containers and iterators are class templates. Iterators are used to step through the elements of a container. Algorithms are used to manipulate data. This chapter discusses some of the containers and iterators. Algorithms are discussed in Chapter 13.

Container Types

Containers are used to manage objects of a given type. The STL containers are classified into three categories:

- Sequence containers (also called sequential containers)
- Associative containers
- Container adapters

Associative containers are described in Chapter 13, and container adapters are described in Chapters 7 and 8.

Sequence Containers

Every object in a sequence container has a specific position. The three predefined sequence containers are as follows:

- `vector`
- `deque`
- `list`

Before discussing container types in general, let us first briefly describe the sequence container `vector`. We do so because vector containers are logically the same as arrays and, therefore, they can be processed like arrays. Also, with the help of vector containers, we can describe several properties that are common to all containers. In fact, all containers use the same names for the common operations. Of course, there are operations that are specific to a container. These operations are discussed when describing a specific container. This chapter discusses `vector` and `deque` containers. Chapter 5 discusses `list` containers.

Sequence Container: `vector`

A vector container stores and manages its objects in a dynamic array. Because an array is a random access data structure, the elements of a vector can be accessed randomly. Item insertion in the middle or beginning of an array is time consuming, especially if the array is large. However, inserting an item at the end is fast.

The name of the class that implements the vector container is `vector`. (Recall that containers are class templates.) The name of the header file containing the `class vector` is `vector`. Thus, to use a vector container in a program, the program must include the following statement:

```
#include <vector>
```

Furthermore, to define an object of type **vector**, we must specify the type of the object because the **class vector** is a class template. For example, the statement

```
vector<int> intList;
```

declares **intList** to be a vector and the component type to be **int**. Similarly, the statement

```
vector<string> stringList;
```

declares **stringList** to be a **vector** container and the component type to be **string**.

DECLARING VECTOR OBJECTS

The **class vector** contains several constructors, including the default constructor. Therefore, a vector container can be declared and initialized in several ways. Table 4-1 describes how a vector container of a specific type can be declared and initialized.

TABLE 4-1 Various ways to declare and initialize a vector container

Statement	Effect
`vector<elementType> vecList;`	Creates an empty vector, **vecList**, without any elements. (The default constructor is invoked.)
`vector<elementType> vecList(otherVecList);`	Creates a vector, **vecList**, and initializes **vecList** to the elements of the vector **otherVecList**. **vecList** and **otherVecList** are of the same type.
`vector<elementType> vecList(size);`	Creates a vector, **vecList**, of size **size**. **vecList** is initialized using the default constructor.
`vector<elementType> vecList(n, elem);`	Creates a vector, **vecList**, of size **n**. **vecList** is initialized using **n** copies of the element **elem**.
`vector<elementType> vecList(begin, end);`	Creates a vector, **vecList**. **vecList** is initialized to the elements in the range **[begin, end)**, that is, all elements in the range **begin...end-1**.

EXAMPLE 4-1

a. The following statement declares `intList` to be an empty vector container and the element type is `int`.

```
vector<int> intList;
```

b. The following statement declares `intList` to be a vector container of size 10 and the element type is `int`. The elements of `intList` are initialized to `0`.

```
vector<int> intList(10);
```

c. The following statement declares `intList` to be a vector container of size 5 and the element type is `int`. The container `intList` is initialized using the elements of the array.

```
int intArray[5] = {2,4,6,8,10};
vector<int> intList(intArray, intArray + 5);
```

The container `intList` is initialized using the elements of the array `intArray`. That is, `intList = {2,4,6,8,10}`.

Now that we know how to declare a vector sequence container, let us now discuss how to manipulate data stored in a vector container. To do so, we must know the following basic operations:

- Item insertion
- Item deletion
- Stepping through the elements of a vector container

The elements in a vector container can be accessed directly by using the operations given in Table 4-2.

TABLE 4-2 Operations to access the elements of a vector container

Expression	Effect
`vecList.at(index)`	Returns the element at the position specified by `index`.
`vecList[index]`	Returns the element at the position specified by `index`.
`vecList.front()`	Returns the first element. (Does not check whether the container is empty.)
`vecList.back()`	Returns the last element. (Does not check whether the container is empty.)

From Table 4-2, it follows that that the elements in a vector can be processed just as they can in an array. (Recall that in C++, arrays start at location 0. Similarly, the first element in a vector container is at location 0.)

EXAMPLE 4-2

Consider the following statement, which declares `intList` to be a vector container of size 5 and the element type is `int`.

```
vector<int> intList(5);
```

You can use a loop, such as the following, to store elements into `intList`:

```
for (int j = 0; j < 5; j++)
    intList[j] = j;
```

Similarly, you can use a `for` loop to output the elements of `intList`.

The `class vector` provides various operations to process the elements of a vector container. Suppose that `vecList` is a container of type `vector`. Item insertion and deletion in `vecList` can be accomplished using the operations given in Table 4-3. These operations are implemented as member functions of the `class vector` and are shown in bold. Table 4-3 also shows how these operations are used.

TABLE 4-3 Various operations on a vector container

Expression	Effect
`vecList.clear()`	Deletes all elements from the container.
`vecList.erase(position)`	Deletes the element at the position specified by position.
`vecList.erase(beg, end)`	Deletes all elements starting at `beg` until `end-1`.
`vecList.insert(position, elem)`	A copy of `elem` is inserted at the position specified by `position`. The position of the new element is returned.
`vecList.insert(position, n, elem)`	`n` copies of `elem` are inserted at the position specified by `position`.
`vecList.insert(position, beg, end)`	A copy of the elements, starting at `beg` until `end-1`, is inserted into `vecList` at the position specified by `position`.

TABLE 4-3 Various operations on a vector container (continued)

Expression	Effect
`vecList.push_back(elem)`	A copy of `elem` is inserted into `vecList` at the end.
`vecList.pop_back()`	Deletes the last element.
`vecList.resize(num)`	Changes the number of elements to `num`. If `size()`, that is, the number of elements in the container increases, the default constructor creates the new elements.
`vecList.resize(num, elem)`	Changes the number of elements to `num`. If `size()` increases, the default constructor creates the new elements.

NOTE

In Table 4-3, the argument `position` in STL terminology is called an **iterator**. An iterator works just like a pointer. In general, iterators are used to step through the elements of a container. In other words, with the help of an iterator, we can walk through the elements of a container and process them one at a time. In the next section, we describe how to declare an iterator in a vector container and how to manipulate the data stored in a container. Because iterators are an integral part of the STL, they are discussed in detail in the section "Iterators," located later in this chapter.

The function `push_back` is quite useful. This function is used to add an element at the end of a container. The container `intList` of size 5 was declared in Example 4-2. You might think that you can only add five elements to the container `intList`. However, this is not the case. If you need to add more than five elements, you can use the function `push_back`. You cannot use the array subscripting operator, as in Example 4-2, to add elements past the position 4 unless you increase the size of the container.

If you do not know the number of elements you need to store in a vector container, then when you declare the vector container you do not need to specify its size (see Example 4-3). In this case, you can use the function `push_back`, as shown in Examples 4-3 and 4-5, to add elements into a vector container.

EXAMPLE 4-3

The following statement declares `intList` to be a vector container of size 0.

```
vector<int> intList;
```

To add elements into `intList`, we can use the function `push_back` as follows:

```
intList.push_back(34);
intList.push_back(55);
```

After these statements execute, the size of `intList` is 2 and `intList = {34, 55}`. Of course, you could have used the `resize` function to increase the size of `intList` and then use the array subscripting operator. However, at times, the `push_back` function is more convenient because it does not need to know the size of the container; it simply adds elements at the end.

Declaring an Iterator to a Vector Container

Even though we can process a vector container just like an array using the array subscripting operator, there are situations where we would like to process the elements of a vector container using an iterator. (Recall that an iterator is just like a pointer.) For example, suppose that we want to insert an element at a specific position in a vector container. Because the element is to be inserted at a specific position, this would require shifting the elements of the container (unless the element is added at the end). Of course, we must also think about the size of the container. To make element(s) insertion convenient, the `class vector` provides the function `insert` to insert the elements at a specific position in a vector container. However, to use the function `insert`, the position where to insert the element(s) must be specified by an iterator. Similarly, the function `erase`, to remove an element, also requires the use of an iterator. This section describes how to declare and use an iterator on a vector container.

The `class vector` contains a `typedef iterator`, which is declared as a `public` member. An iterator to a vector container is declared using the `typedef iterator`. For example, the statement

```
vector<int>::iterator intVecIter;
```

declares `intVecIter` to be an iterator into a vector container of type `int`.

Because `iterator` is a `typedef` defined inside the `class vector`, we must use the container name (`vector`), container element type, and scope resolution operator to use the `typedef iterator`.

Suppose that the iterator `intVecIter` points to an element of a vector container whose elements are of type `int`. The expression

```
++intVecIter
```

advances the iterator `intVecIter` to the next element into the container. The expression

```
*intVecIter
```

returns the element at the current iterator position.

Note that these operations are the same as the operations on pointers, discussed in Chapter 3. Recall that when used as a unary operator, `*` is called the dereferencing operator.

We now discuss how to use an iterator into a vector container to manipulate the data stored into it. Suppose that we have the following statements:

```
vector<int> intList;                    //Line 1
vector<int>::iterator intVecIter;       //Line 2
```

The statement in Line 1 declares `intList` to be a `vector` container and the element type to be `int`. The statement in Line 2 declares `intVecIter` to be an iterator into a `vector` container whose element type is `int`.

Containers and the Functions `begin` and `end`

Every container contains the member functions `begin` and `end`. The function `begin` returns the position of the first element into the container; the function `end` returns the position of the last element into the container. These functions have no parameters.

After the following statement executes:

```
intVecIter = intList.begin();
```

the iterator `intVecIter` points to the first element into the container `intList`.

The following `for` loop uses an iterator to output the elements of `intList` onto the standard output device:

```
for (intVecIter = intList.begin(); intVecIter != intList.end();
                                   intVecList)
    cout << *intVecIter << " ";
```

EXAMPLE 4-4

Consider the following statements:

```
int intArray[7] = {1, 3, 5, 7, 9, 11, 13};    //Line 1
vector<int> vecList(intArray, intArray + 7};  //Line 2
vector<int>::iterator  intVecIter;            //Line 3
```

The statement in Line 2 declares and initializes the vector container `vecList`. Now consider the following statements:

```
intVecIter = vecList.begin();     //Line 4
++intVecIter;                     //Line 5
vecList.insert(intVecIter, 22};   //Line 6
```

The statement in Line 4 initializes the iterator `intVecIter` to the first element of `vecList`. The statement in Line 5 advances `intVecIter` to the second element of `vecList`. The statement in Line 6 inserts `22` at the position specified by `intVecIter`. After the statement in Line 6 executes, `vecList = {1, 22, 3, 5, 7, 9, 11, 13}`. Notice that the size of the container also increases.

The `class vector` also contains member functions that can be used to find the number of elements currently in the container, the maximum number of elements that can be inserted in a container, and so on. Table 4-4 describes some of these operations. (Suppose that `vecCont` is a vector container.)

TABLE 4-4 Functions to determine the size of a vector container

Expression	Effect
`vecCont.capacity()`	Returns the maximum number of elements that can be inserted into the container `vecCont` without reallocation.
`vecCont.empty()`	Returns `true` if the container `vecCont` is empty and `false` otherwise.
`vecCont.size()`	Returns the number of elements currently in the container `vecCont`.
`vecCont.max_size()`	Returns the maximum number of elements that can be inserted into the container `vecCont`.

Example 4-5 illustrates how to use a vector container in a program and how to process the elements into a vector container.

EXAMPLE 4-5

```
//***********************************************************
// Author: D.S. Malik
//
// This program illustrates how to use a vector container in a
// program.
//***********************************************************

#include <iostream>                                    //Line 1
#include <vector>                                      //Line 2

using namespace std;                                   //Line 3
```

```
int main()                                                  //Line 4
{                                                           //Line 5
    vector<int>  intList;                                   //Line 6

    intList.push_back(13);                                  //Line 7
    intList.push_back(75);                                  //Line 8
    intList.push_back(28);                                  //Line 9
    intList.push_back(35);                                  //Line 10

    cout << "Line 11: List Elements: ";                     //Line 11
    for (int i = 0; i < 4; i++)                             //Line 12
        cout << intList[i] << " ";                          //Line 13
    cout << endl;                                           //Line 14

    for (int i = 0; i < 4; i++)                             //Line 15
        intList[i] *= 2;                                    //Line 16

    cout << "Line 17: List Elements: ";                     //Line 17
    for (int i = 0; i < 4; i++)                             //Line 18
        cout << intList[i] << " ";                          //Line 19
    cout << endl;                                           //Line 20

    vector<int>::iterator  listIt;                          //Line 21

    cout << "Line 22: List Elements: ";                     //Line 22
    for (listIt = intList.begin();
                 listIt != intList.end(); ++listIt)         //Line 23
        cout << *listIt << " ";                             //Line 24
    cout << endl;                                           //Line 25

    listIt = intList.begin();                               //Line 26
    ++listIt;                                               //Line 27
    ++listIt;                                               //Line 28
    intList.insert(listIt, 88);                             //Line 29

    cout << "Line 30: List Elements: ";                     //Line 30
    for (listIt = intList.begin();
                 listIt != intList.end(); ++listIt)         //Line 31
        cout << *listIt << " ";                             //Line 32
    cout << endl;                                           //Line 33

    return 0;                                               //Line 34
}                                                           //Line 35
```

Sample Run:

```
Line 11: List Elements: 13 75 28 35
Line 17: List Elements: 26 150 56 70
Line 22: List Elements: 26 150 56 70
Line 30: List Elements: 26 150 88 56 70
```

The statement in Line 6 declares a vector container (or vector for short), `intList`, of type `int`. The statements in Lines 7 through 10 use the operation `push_back` to insert four numbers—`13`, `75`, `28`, and `35`—into `intList`. The statements in Lines 12 and 13 use the `for` loop and the array subscripting operator `[]` to output the elements of `intList`. In the output, see the line marked Line 11, which contains the output of Lines 11 through 14 of the program. The statements in Lines 15 and 16 use a `for` loop to double the value of each element of `intList`; the statements in Lines 18 and 19 output the elements of `intList`. In the output, see the line marked Line 17, which contains the output of Lines 17 through 20 of the program.

The statement in Line 21 declares `listIt` to be a vector iterator that processes any vector container whose elements are of type `int`. Using the iterator `listIt`, the statements in Lines 23 and 24 output the elements of `intList`. After the statement in Line 26 executes, `listIt` points to the first element of `intList`. The statements in Lines 27 and 28 advance `listIt` twice; after these statements execute, `listIt` points to the third element of `intList`. The statement in Line 29 inserts `88` into `intList` at the position specified by the iterator `listIt`. Because `listIt` points to the component at position 2 (the third element of `intList`), `88` is inserted at position 2 in `intList`; that is, `88` becomes the third element of `intList`. The statements in Lines 31 and 32 output the modified `intList`.

Member Functions Common to All Containers

The previous section discussed vector containers. We now look at operations that are common to all containers. For example, every container class has a default constructor, several constructors with parameters, a destructor, a function to insert an element into a container, and so on.

Recall that a class encapsulates data, and operations on that data, into a single unit. Because every container is a class, several operations are directly defined for a container and are provided as part of the class definition. Also, recall that the operations to manipulate the data are implemented with the help of functions and are called member functions of the class. Table 4-5 describes the member functions that are common to all containers; that is, these functions are included as members of the class template implementing the container.

Suppose that `ct`, `ct1`, and `ct2` are containers of the same type. Table 4-5 shows the name of the function in bold, and shows how a function is called.

TABLE 4-5 Member functions common to all containers

Member function	Effect
Default constructor	Initializes the object to an empty state.
Constructor with parameters	In addition to the default constructor, every container has constructors with parameters. We describe these constructors when we discuss a specific container.
Copy constructor	Executes when an object is passed as a parameter by value, and when an object is declared and initialized using another object of the same type.
Destructor	Executes when the object goes out of scope.
`ct.empty()`	Returns `true` if container `ct` is empty and `false` otherwise.
`ct.size()`	Returns the number of elements currently in container `ct`.
`ct.max_size()`	Returns the maximum number of elements that can be inserted into container `ct`.
`ct1.swap(ct2)`	Swaps the elements of containers `ct1` and `ct2`.
`ct.begin()`	Returns an iterator to the first element into container `ct`.
`ct.end()`	Returns an iterator to the last element into container `ct`.
`ct.rbegin()`	Reverse begin. Returns a pointer to the last element into container `ct`. This function is used to process the elements of `ct` in reverse.
`ct.rend()`	Reverse end. Returns a pointer to the first element into container `ct`.
`ct.insert(position, elem)`	Inserts `elem` into container `ct` at the position specified by the argument `position`. Note that here `position` is an iterator.
`ct.erase(begin, end)`	Deletes all elements between `begin...end-1` from container `ct`.

TABLE 4-5 Member functions common to all containers (continued)

Member function	Effect
`ct.clear()`	Deletes all elements from the container. After a call to this function, container `ct` is empty.
Operator functions	
`ct1 = ct2`	Copies the elements of `ct2` into `ct1`. After this operation, the elements in both containers are the same.
`ct1 == ct2`	Returns `true` if containers `ct1` and `ct2` are equal and `false` otherwise.
`ct1 != ct2`	Returns `true` if containers `ct1` and `ct2` are not equal and `false` otherwise.
`ct1 < ct2`	Returns `true` if container `ct1` is less than container `ct2` and `false` otherwise.
`ct1 <= ct2`	Returns `true` if container `ct1` is less than or equal to container `ct2` and `false` otherwise.
`ct1 > ct2`	Returns `true` if container `ct1` is greater than container `ct2` and `false` otherwise.
`ct1 >= ct2`	Returns `true` if container `ct1` is greater than or equal to container `ct2` and `false` otherwise.

NOTE Because these operations are common to all containers, when discussing a specific container, to save space, these operations are not listed again.

Member Functions Common to Sequence Containers

The previous section described the member functions that are common to all containers. In addition to these member functions, Table 4-6 describes the member functions that are common to all sequence containers—that is, containers of type `vector`, `deque`, and `list`. (Suppose that `seqCont` is a sequence container.)

TABLE 4-6 Member functions common to all sequence containers

Expression	Effect
`seqCont.insert(position, elem)`	A copy of `elem` is inserted at the position specified by `position`. The position of the new element is returned.
`seqCont.insert(position, n, elem)`	`n` copies of `elem` are inserted at the position specified by `position`.
`seqCont.insert(position, beg, end)`	A copy of the elements, starting at `beg` until `end-1`, are inserted into `seqCont` at the position specified by `position`.
`seqCont.push_back(elem)`	A copy of `elem` is inserted into `seqCont` at the end.
`seqCont.pop_back()`	Deletes the last element.
`seqCont.erase(position)`	Deletes the element at the position specified by `position`.
`seqCont.erase(beg, end)`	Deletes all elements starting at `beg` until `end-1`.
`seqCont.clear()`	Deletes all elements from the container.
`seqCont.resize(num)`	Changes the number of elements to `num`. If `size()` grows, the new elements are created by their default constructor.
`seqCont.resize(num, elem)`	Changes the number of elements to `num`. If `size()` grows, the new elements are copies of `elem`.

The `copy` Algorithm

Example 4-5 used a `for` loop to output the elements of a vector container. The STL provides a convenient way to output the elements of a container with the help of the function `copy`. The function `copy` is provided as a part of the generic algorithms of the STL and can be used with any container type as well as arrays. Because we frequently need to output the elements of a container, before continuing with our discussion of containers, let us describe this function.

NOTE Like the function copy, the STL contains many functions as part of generic algorithms, which are described in Chapter 13.

The function copy does more than output the elements of a container. In general, it allows us to copy the elements from one place to another. For example, to output the elements of a vector or to copy the elements of a vector into another vector, we can use the function copy. The prototype of the function template copy is as follows:

```
template <class inputIterator, class outputIterator>
outputItr copy(inputIterator first1, inputIterator last,
               outputIterator first2);
```

The parameter first1 specifies the position from which to begin copying the elements; the parameter last specifies the end position. The parameter first2 specifies where to copy the elements. Therefore, the parameters first1 and last specify the source, and the parameter first2 specifies the destination. Note that the elements within the range first1...last-1 are copied.

The definition of the function template copy is contained in the header file algorithm. Thus, to use the function copy, the program must include the statement

```
#include <algorithm>
```

The function copy works as follows. Consider the following statement:

```
int intArray[] = {5, 6, 8, 3, 40, 36, 98, 29, 75};   //Line 1
vector<int> vecList(9);                              //Line 2
```

This statement in Line 1 creates the array intArray of nine components—that is,

```
intArray = {5, 6, 8, 3, 40, 36, 98, 29, 75}
```

Here intArray[0] = 5, intArray[1] = 6, and so on.

The statement in Line 2 creates an empty container of nine components of type vector and the element type int.

Recall that the array name, intArray, is actually a pointer and contains the base address of the array. Therefore, intArray points to the first component of the array, intArray + 1 points to the second component of the array, and so on.

Now consider the statement

```
copy(intArray, intArray+9, vecList.begin());   //Line 3
```

This statement copies the elements starting at the location intArray, which is the first component of the array intArray, until intArray + 9 - 1 (that is, intArray + 8), which is the last element of the array intArray, into the container vecList. (Note that here first1 is intArray, last is intArray + 9, and first2 is vecList.begin().) After the statement in Line 3 executes,

```
vecList = {5, 6, 8, 3, 40, 36, 98, 29, 75}  //Line 4
```

Next, consider the statement

```
copy(intArray + 1, intArray + 9, intArray);   //Line 5
```

Here `first1` is `intArray + 1`; that is, `first1` points to the location of the second element of the array `intArray`, and `last` is `intArray + 9`. Also, `first2` is `intArray`; that is, `first2` points to the location of the first element of the array `intArray`. Therefore, the second array element is copied into the first array component, the third array element into the second array component, and so on. After the statement in Line 5 executes,

```
intArray[] = {6, 8, 3, 40, 36, 98, 29, 75, 75}    //Line 6
```

Notice that the elements of the array `intArray` are shifted to the left by one position.

Suppose that `vecList` is as in Line 4. Consider the statement

```
copy(vecList.rbegin() + 2, vecList.rend(),
                           vecList.rbegin());  //Line 7
```

Recall that the function `rbegin` (reverse begin) returns a pointer to the last element into a container; it is used to process the elements of a container in reverse. Therefore, `vecList.rbegin() + 2` returns a pointer to the third-to-last element into the container `vecList`. Similarly, the function `rend` (reverse end) returns a pointer to the first element of a container. The previous statement shifts the elements of the container `vecList` to the right by two positions. After the statement in Line 7 executes, the container `vecList` is as follows:

```
vecList = {5, 6, 5, 6, 8, 3, 40, 36, 98}
```

Example 4-6 shows the effect of the preceding statements using a C++ program. Before discussing Example 4-6, let us describe a special type of iterator, called **`ostream` iterators**, which work well with the function `copy` to copy the elements of a container to an output device.

`ostream` Iterator and Function `copy`

One way to output the contents of a container is to use a **`for`** loop and the function `begin` to initialize the **`for`** loop control variable, and to use the function `end` to set the limit. Alternatively, the function `copy` can be used to output the elements of a container. In this case, an iterator of type `ostream` specifies the destination (`ostream` iterators are discussed in detail later in this chapter). When we create an iterator of type `ostream`, we also specify the type of element the iterator will output.

The following statement illustrates how to create an `ostream` iterator of type **`int`**:

```
ostream_iterator<int> screen(cout, " "); //Line A
```

This statement creates `screen` to be an `ostream` iterator with the element type `int`. The iterator `screen` has two arguments: the object `cout` and a space. Thus, the iterator `screen` is initialized using the object `cout`, and when this iterator outputs the elements they are separated by a space.

The statement

```
copy(intArray, intArray+9, screen);
```

outputs the elements of `intArray` on the screen.

Similarly, the statement

```
copy(vecList.begin(), vecList.end(), screen);
```

outputs the elements of the container `vecList` on the screen.

We will frequently use the function `copy` to output the elements of a container by using an `ostream` iterator. Also, until we discuss `ostream` iterators in detail, we will use statements similar to the statement in Line A to create an `ostream` iterator.

Of course, we can directly specify an `ostream` iterator in the function `copy`. For example, the statement (shown previously)

```
copy(vecList.begin(), vecList.end(), screen);
```

is equivalent to the statement

```
copy(vecList.begin(), vecList.end(), ostream_iterator<int>(cout, " "));
```

Finally, the statement

```
copy(vecList.begin(), vecList.end(),
                      ostream_iterator<int>(cout, ", "));
```

outputs the elements of `vecList` with a comma and space between them.

Example 4-6 illustrates how to use the function `copy` and an `ostream` iterator in a program.

EXAMPLE 4-6

```
//***********************************************************
// Author: D.S. Malik
//
// This program illustrates how to use the function copy and
// an ostream iterator in a program.
//***********************************************************

#include <algorithm>                                    //Line 1
#include <vector>                                       //Line 2
#include <iterator>                                     //Line 3
#include <iostream>                                     //Line 4
```

```
using namespace std;                                          //Line 5

int main()                                                    //Line 6
{                                                             //Line 7
    int intArray[] = {5, 6, 8, 3, 40, 36, 98, 29, 75};        //Line 8

    vector<int> vecList(9);                                   //Line 9

    ostream_iterator<int> screen(cout, " ");                  //Line 10

    cout << "Line 11: intArray: ";                            //Line 11
    copy(intArray, intArray + 9, screen);                     //Line 12
    cout << endl;                                             //Line 13

    copy(intArray, intArray + 9, vecList.begin());            //Line 14

    cout << "Line 15: vecList: ";                             //Line 15
    copy(vecList.begin(), vecList.end(), screen);             //Line 16
    cout << endl;                                             //Line 17

    copy(intArray + 1, intArray + 9, intArray);               //Line 18
    cout << "Line 19: After shifting the elements one "
         << "position to the left, intArray: " << endl;       //Line 19
    copy(intArray, intArray + 9, screen);                     //Line 20
    cout << endl;                                             //Line 21

    copy(vecList.rbegin() + 2, vecList.rend(),
                               vecList.rbegin());             //Line 22
    cout << "Line 23: After shifting the elements down "
         << "by two positions, vecList:" << endl;             //Line 23
    copy(vecList.begin(), vecList.end(), screen);             //Line 24
    cout << endl;                                             //Line 25

    return 0;                                                 //Line 26
}                                                             //Line 27
```

Sample Run:

```
Line 11: intArray: 5 6 8 3 40 36 98 29 75
Line 15: vecList: 5 6 8 3 40 36 98 29 75
Line 19: After shifting the elements one position to the left, intArray:
6 8 3 40 36 98 29 75 75
Line 23: After shifting the elements down by two positions, vecList:
5 6 5 6 8 3 40 36 98
```

Sequence Container: `deque`

This section describes the `deque` sequence containers. The term **deque** stands for double-ended queue. Deque containers are implemented as dynamic arrays in such a way that the elements can be inserted at both ends. Thus, a `deque` can expand in either

direction. Elements can also be inserted in the middle. Inserting elements at the beginning or at the end is fast; inserting elements in the middle, however, is time consuming because the elements in the queue need to be shifted.

The name of the class defining the deque containers is `deque`. The definition of the `class deque`, and the functions to implement the various operations on a `deque` object, are also contained in the header file `deque`. Therefore, to use a `deque` container in a program, the program must include the following statement:

```
#include <deque>
```

The `class deque` contains several constructors. Thus, a `deque` object can be initialized in various ways when it is declared, as described in Table 4-7.

TABLE 4-7 Various ways to declare a `deque` object

Statement	Effect
`deque<elementType> deq;`	Creates an empty `deque` container without any elements. (The default constructor is invoked.)
`deque<elementType> deq(otherDeq);`	Creates a deque container, `deq`, and initializes `deq` to the elements of `otherDeq`; `deq` and `otherDeq` are of the same type.
`deque<elementType> deq(size);`	Creates a deque container, `deq`, of size `size`. `deq` is initialized using the default constructor.
`deque<elementType> deq(n, elem);`	Creates a deque container, `deq`, of size `n`. `deq` is initialized using `n` copies of the element `elem`.
`deque<elementType> deq(begin, end);`	Creates a deque container, `deq`. `deq` is initialized to the elements in the range `[begin, end)`—that is, all elements in the range `begin...end-1`.

In addition to the operations that are common to all containers (see Table 4-6), Table 4-8 describes the operations that can be used to manipulate the elements of a `deque` container. The name of the function implementing the operations is shown in bold. The statement also shows how to use a particular function. Suppose that `deq` is a `deque` container.

TABLE 4-8 Various operations that can be performed on a `deque` object

Expression	Effect
`deq.assign(n,elem)`	Assigns `n` copies of `elem`.
`deq.assign(beg,end)`	Assigns all the elements in the range `beg...end-1`.
`deq.push_front(elem)`	Inserts `elem` at the beginning of `deq`.
`deq.pop_front()`	Removes the first element from `deq`.
`deq.at(index)`	Returns the element at the position specified by `index`.
`deq[index]`	Returns the element at the position specified by `index`.
`deq.front()`	Returns the first element. (Does not check whether the container is empty.)
`deq.back()`	Returns the last element. (Does not check whether the container is empty.)

Example 4-7 illustrates how to use a `deque` container in a program.

EXAMPLE 4-7

```
//***********************************************************
// Author: D.S. Malik
//
// This program illustrates how to use a deque container in a
// program.
//***********************************************************

#include <iostream>                                          //Line 1
#include <deque>                                             //Line 2
#include <algorithm>                                         //Line 3
#include <iterator>                                          //Line 4

using namespace std;                                         //Line 5

int main()                                                   //Line 6
{                                                            //Line 7
    deque<int>  intDeq;                                      //Line 8
    ostream_iterator<int> screen(cout, " ");                 //Line 9

    intDeq.push_back(13);                                    //Line 10
    intDeq.push_back(75);                                    //Line 11
```

```
    intDeq.push_back(28);                                   //Line 12
    intDeq.push_back(35);                                   //Line 13

    cout << "Line 14: intDeq: ";                            //Line 14
    copy(intDeq.begin(), intDeq.end(), screen);             //Line 15
    cout << endl;                                           //Line 16

    intDeq.push_front(0);                                   //Line 17
    intDeq.push_back(100);                                  //Line 18

    cout << "Line 19: After adding two more "
         << "elements, one at the front " << endl
         << "         and one at the back, intDeq: ";      //Line 19
    copy(intDeq.begin(), intDeq.end(), screen);             //Line 20
    cout << endl;                                           //Line 21

    intDeq.pop_front();                                     //Line 22
    intDeq.pop_front();                                     //Line 23

    cout << "Line 24: After removing the first "
         << "two elements, intDeq: ";                       //Line 24
    copy(intDeq.begin(), intDeq.end(), screen);             //Line 25
    cout << endl;                                           //Line 26

    intDeq.pop_back();                                      //Line 27
    intDeq.pop_back();                                      //Line 28

    cout << "Line 29: After removing the last "
         << "two elements, intDeq = ";                      //Line 29
    copy(intDeq.begin(), intDeq.end(), screen);             //Line 30
    cout << endl;                                           //Line 31

    deque<int>::iterator  deqIt;                            //Line 32

    deqIt = intDeq.begin();                                 //Line 33
    ++deqIt;   //deqIt points to the second element       //Line 34
    intDeq.insert(deqIt, 444);                              //Line 35
    cout << "Line 36: After inserting 444, intDeq:  ";      //Line 36
    copy(intDeq.begin(), intDeq.end(), screen);             //Line 37
    cout << endl;                                           //Line 38

    return 0;                                               //Line 39
}                                                           //Line 40
```

Sample Run:

```
Line 14:  intDeq: 13 75 28 35
Line 19: After adding two more elements, one at the front
         and one at the back, intDeq: 0 13 75 28 35 100
Line 24: After removing the first two elements, intDeq: 75 28 35 100
Line 29: After removing the last two elements, intDeq = 75 28
Line 36: After inserting 444, intDeq:  75 444 28
```

The statement in Line 8 declares a `deque` container `intDeq` of type `int`; that is, all the elements of `intDeq` are of type `int`. The statement in Line 9 declares `screen` to be an `ostream` iterator initialized to the standard output device. The statements in Lines 10 through 13 use the `push_back` operation to insert four numbers—`13`, `75`, `28`, and `35`—into `intDeq`. The statement in Line 15 outputs the elements of `intDeq`. In the output, see the line marked Line 14, which contains the output of the statements in Lines 14 through 16 of the program.

The statement in Line 17 inserts `0` at the beginning of `intDeq`. The statement in Line 18 inserts `100` at the end of `intDeq`. The statement in Line 20 outputs the modified `intDeq`.

The statements in Lines 22 and 23 use the operation `pop_front` to remove the first two elements of `intDeq`; the statement in Line 25 outputs the modified `intDeq`. The statements in Lines 27 and 28 use the operation `pop_back` to remove the last two elements of `intDeq`; the statement in Line 30 outputs the modified `intDeq`.

The statement in Line 32 declares `deqIt` to be a `deque` iterator that processes all `deque` containers whose elements are of type `int`. After the statement in Line 33 executes, `deqIt` points to the first element of `intDeq`. The statement in Line 34 advances `deqIt` to the next element of `intDeq`. The statement in Line 35 inserts `444` into `intDeq` at the position specified by `deqIt`. The statement in Line 37 outputs `intDeq`.

Iterators

Examples 4-5 through 4-7 further clarify that iterators are quite important to efficiently process the elements of a container. Let us discuss iterators in some detail.

Iterators work just like pointers. In general, an iterator points to the elements of a container (sequence or associative). Thus, with the help of iterators, we can successively access each element of a container.

The two most common operations on iterators are `++` (the increment operator) and `*` (the dereferencing operator). Suppose that `cntItr` is an iterator into a container. The statement

```
++cntItr;
```

advances `cntItr` so that it points to the next element in the container. The statement

```
*cntItr;
```

returns the value of the element of the container pointed to by `cntItr`.

Types of Iterators

There are five types of iterators: input iterators, output iterators, forward iterators, bidirectional iterators, and random access iterators. In the next few sections, we describe these iterators.

Input Iterators

Input iterators, with read access, step forward element-by-element and so return the values element-by-element. These iterators are provided for reading data from an input stream.

Suppose `inputIterator` is an input iterator. Table 4-9 describes the operations on `inputIterator`.

TABLE 4-9 Operations on an input iterator

Expression	Effect
`*inputIterator`	Gives access to the element to which `inputIterator` points.
`inputIterator->member`	Gives access to the member of the element.
`++inputIterator`	Moves forward, returns the new position (preincrement).
`inputIterator++`	Moves forward, returns the old position (postincrement).
`inputIt1 == inputIt2`	Returns `true` if the two iterators are the same and `false` otherwise.
`inputIt1 != inputIt2`	Returns `true` if the two iterators are not the same and `false` otherwise.
`Type(inputIterator)`	Copies the iterators.

Output Iterators

Output iterators, with write access, step forward element-by-element. These iterators are typically used for writing data to an output stream.

Suppose `outputIterator` is an output iterator. Table 4-10 describes the operations on `outputIterator`.

TABLE 4-10 Operations on an output iterator

Expression	Effect
`*outputIterator = value;`	Writes the `value` at the position specified by the `outputIterator`.
`++outputIterator`	Moves forward, returns the new position (preincrement).
`outputIterator++`	Moves forward, returns the old position (postincrement).
`Type(outputIterator)`	Copies the iterators.

NOTE Output iterators cannot be used to iterate over a range twice. Thus, if we write data at the same position twice, there is no guarantee that the new value will replace the old value.

Forward Iterators

Forward iterators combine all of the functionality of input iterators and almost all of the functionality of output iterators. Suppose `forwardIterator` is a forward iterator. Table 4-11 describes the operations on `forwardIterator`.

TABLE 4-11 Operations on a forward iterator

Expression	Effect
`*forwardIterator`	Gives access to the element to which `forwardIterator` points.
`forwardIterator->member`	Gives access to the member of the element.
`++forwardIterator`	Moves forward, returns the new position (preincrement).
`forwardIterator++`	Moves forward, returns the old position (postincrement).
`forwardIt1 == forwardIt2`	Returns `true` if the two iterators are the same and `false` otherwise.
`forwardIt1 != forwardIt2`	Returns `true` if the two iterators are not the same and `false` otherwise.
`forwardIt1 = forwardIt2`	Assignment.

NOTE A forward iterator can refer to the same element in the same collection and process the same element more than once.

Bidirectional Iterators

Bidirectional iterators are forward iterators that can also iterate backward over the elements. Suppose `biDirectionalIterator` is a bidirectional iterator. The operations defined for forward iterators (Table 4-11) are also applicable to bidirectional iterators. To step backward, the decrement operations are also defined for `biDirectionalIterator`. Table 4-12 shows additional operations on a bidirectional iterator.

TABLE 4-12 Additional operations on a bidirectional iterator

Expression	Effect
`--biDirectionalIterator`	Moves backward, returns the new position (predecrement).
`biDirectionalIterator--`	Moves backward, returns the old position (postdecrement).

NOTE Bidirectional iterators can be used only with containers of type `vector`, `deque`, `list`, `set`, `multiset`, `map`, and `multimap`.

Random Access Iterators

Random access iterators are bidirectional iterators that can randomly process the elements of a container. These iterators can be used with containers of type `vector`, `deque`, `string`, and arrays. The operations defined for bidirectional iterators (for example, Tables 4-11 and 4-12) are also applicable to random access iterators. Table 4-13 describes the additional operations that are defined for random access iterators. (Suppose `rAccessIterator` is a random access iterator.)

TABLE 4-13 Additional operations on a random access iterator

Expression	Effect
`rAccessIterator[n]`	Accesses the `n`th element.
`rAccessIterator += n`	Moves `rAccessIterator` forward `n` elements if `n >= 0` and backward if `n < 0`.
`rAccessIterator -= n`	Moves `rAccessIterator` backward `n` elements if `n >= 0` and forward if `n < 0`.
`rAccessIterator + n`	Returns the iterator of the next `n`th element.
`n + rAccessIterator`	Returns the iterator of the next `n`th element.
`rAccessIterator - n`	Returns the iterator of the previous `n`th element.
`rAccessIt1 - rAccessIt2`	Returns the distance between the iterators `rAccessIt1` and `rAccessIt2`.
`rAccessIt1 < rAccessIt2`	Returns `true` if `rAccessIt1` is before `rAccessIt2` and `false` otherwise.
`rAccessIt1 <= rAccessIt2`	Returns `true` if `rAccessIt1` is before or equal to `rAccessIt2` and `false` otherwise.
`rAccessIt1 > rAccessIt2`	Returns `true` if `rAccessIt1` is after `rAccessIt2` and `false` otherwise.
`rAccessIt1 >= rAccessIt2`	Returns `true` if `rAccessIt1` is after or equal to `rAccessIt2` and `false` otherwise.

Figure 4-1 shows the iterator hierarchy.

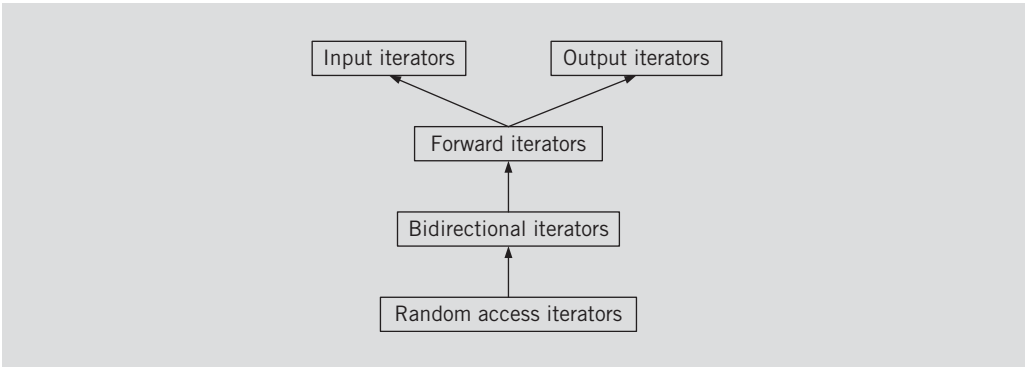


FIGURE 4-1 Iterator hierarchy

Now that you know the different types of iterators, next we describe how to declare an iterator to a container.

`typedef iterator` Every container (sequence or associative) contains a `typedef iterator`. Thus, an iterator into a container is declared using the `typedef iterator`. For example, the statement

```
vector<int>::iterator intVecIter;
```

declares `intVecIter` to be an iterator into a `vector` container of type `int`. Moreover, the iterator `intVecIter` can be used on any `vector<int>`, but not on any other container, such as `vector<double>`, `vector<string>`, and `deque`.

Because `iterator` is a `typedef` defined inside a container (that is, a class) such as `vector`, we must use the appropriate container name, container element type, and the scope resolution operator to use the `typedef iterator`.

`typedef const_iterator` Because an iterator works like a pointer, with the help of an iterator into a container and the dereferencing operator, `*`, we can modify the elements of the container. However, if a container is declared as **`const`**, then we must prevent the iterator from modifying the elements of the container, especially accidentally. To handle this situation, every container contains another **`typedef`** `const_iterator`. For example, the statement

```
vector<int>::const_iterator  intConstVecIt;
```

declares `intConstVecIt` to be an iterator into a `vector` container whose elements are of type **`int`**. The iterator `intConstVecIt` is used to process the elements of those vector containers that are declared as constant vector containers of type `vector<int>`.

An iterator of type `const_iterator` is a read-only iterator.

typedef reverse_iterator Every container also contains the **typedef reverse_iterator**. An iterator of this type is used to iterate through the elements of a container in reverse.

typedef const_reverse_iterator An iterator of this type is a read-only iterator and is used to iterate through the elements of a container in reverse. It is required if the container is declared as **const** and we need to iterate through the elements of the container in reverse.

In addition to the previous four **typedef**s, several other **typedef**s are common to all containers. Table 4-14 describes them.

TABLE 4-14 Various typedefs common to all containers

typedef	Effect
difference_type	The type of result from subtracting two iterators referring to the same container.
pointer	A pointer to the type of elements stored in the container.
reference	A reference to the type of elements stored in the container.
const_reference	A constant reference to the type of elements stored in the container. A constant reference is read-only.
size_type	The type used to count the elements in a container. This type is also used to index through sequence containers, except **list** containers.
value_type	The type of container elements.

Stream Iterators

Another useful set of iterators is stream iterators—**istream** iterators and **ostream** iterators. This section describes both types of iterators.

istream_iterator The **istream** iterator is used to input data into a program from an input stream. The **class istream_iterator** contains the definition of an input stream iterator. The general syntax to use an **istream** iterator is as follows:

```
istream_iterator<Type> isIdentifier(istream&);
```

where `Type` is either a built-in type or a user-defined class type, for which an input iterator is defined. The identifier `isIdentifier` is initialized using the constructor whose argument is either an `istream` class object such as `cin`, or any publicly defined `istream` subtype, such as `ifstream`.

ostream_iterator The `ostream` iterators are used to output data from a program into an output stream. These iterators were defined earlier in this chapter. We review them here for the sake of completeness.

The `class ostream_iterator` contains the definition of an output stream iterator. The general syntax to use an `ostream` iterator is as follows:

```
ostream_iterator<Type> osIdentifier(ostream&);
```

or

```
ostream_iterator<Type> osIdentifier(ostream&, char* deLimit);
```

where `Type` is either a built-in type or a user-defined class type, for which an output iterator is defined. The identifier `osIdentifier` is initialized using the constructor whose argument is either an `ostream` class object such as `cout`, or any publicly defined `ostream` subtype, such as `ofstream`. In the second form used for declaring an `ostream` iterator, by using the second argument (`deLimit`) of the initializing constructor, we can specify a character separating the output.

PROGRAMMING EXAMPLE: Grade Report

The midsemester point at your local university is approaching. The registrar's office wants to prepare the grade reports as soon as the students' grades are recorded. Some of the students enrolled have not yet paid their tuition, however.

If a student has paid the tuition, the grades are shown on the grade report together with the grade point average (GPA). If a student has not paid the tuition, the grades are not printed. For these students, the grade report contains a message indicating that the grades have been held for nonpayment of the tuition. The grade report also shows the billing amount.

The registrar's office and the business office want your help in writing a program that can analyze the students' data and print the appropriate grade reports. The data is stored in a file in the following form:

```
345
studentName studentID isTuitionPaid numberOfCourses
courseName courseNumber creditHours grade
```

```
courseName courseNumber creditHours grade
...
studentName studentID isTuitionPaid numberOfCourses
courseName courseNumber creditHours grade
courseName courseNumber creditHours grade
...
```

The first line indicates the tuition rate per credit hour. The students' data is given thereafter.

A sample input file follows:

```
345
Lisa Miller 890238 Y 4
Mathematics MTH345 4 A
Physics PHY357 3 B
ComputerSci CSC478 3 B
History HIS356 3 A
...
```

The first line indicates that the tuition rate is $345 per credit hour. Next, the course data for student `Lisa Miller` is given: Lisa Miller's ID is `890238`, she has paid the tuition, and is taking `4` courses. The course number for the mathematics class she is taking is MTH345, the course has `4` credit hours, her midsemester grade is `A`, and so on.

The desired output for each student is in the following form:

```
Student Name: Lisa Miller
Student ID: 890238
Number of courses enrolled: 4

Course No  Course Name  Credits Grade
CSC478     ComputerSci      3      B
HIS356     History          3      A
MTH345     Mathematics      4      A
PHY357     Physics          3      B

Total number of credits: 13
Midsemester GPA: 3.54
```

This output shows that the courses must be ordered according to the course number. To calculate the GPA, we assume that the grade `A` is equivalent to `4` points, `B` is equivalent to `3` points, `C` is equivalent to `2` points, `D` is equivalent to `1` point, and `F` is equivalent to `0` points.

Input A file containing the data in the form given previously. For easy reference in the rest of the discussion, let us assume that the name of the input file is `stData.txt`.

Output A file containing the output of the form given previously. Let us assume that the name of the output file is `stDataOut.txt`.

PROBLEM ANALYSIS AND ALGORITHM DESIGN

We must first identify the main components of the program. The university has students, and every student takes courses. Thus, the two main components are the student and the course.

Let us first describe the component course.

Course

The main characteristics of a course are the course name, course number, and number of credit hours. Although the grade a student receives is not really a characteristic of a course, to simplify the program this component also includes the student's grade.

Some of the basic operations that need to be performed on an object of the course type are as follows:

1. Set the course information.
2. Print the course information.
3. Show the credit hours.
4. Show the course number.
5. Show the grade.

The following class defines the course as an ADT:

```
class courseType
{
public:
    void setCourseInfo(string cName, string cNo,
                       char grade, int credits);
      //Function to set the course information
      //The course information is set according to the
      //incoming parameters.
      //Postcondition: courseName = cName; courseNo = cNo;
      //     courseGrade = grade; courseCredits = credits;

    void print(ostream& outp, bool isGrade);
      //Function to print the course information
      //If the bool parameter isGrade is true, the grade is
      //shown, otherwise three stars are printed.

    int getCredits();
      //Function to return the credit hours
      //The value of the private data member courseCredits
      //is returned.

    void getCourseNumber(string& cNo);
      //Function to return the course number
      //Postcondition:  cNo = courseNo;

    char getGrade();
      //Function to return the grade for the course
      //The value of the private data member courseGrade
      //is returned.
```

```
    bool operator==(const courseType&) const;
    bool operator!=(const courseType&) const;
    bool operator<=(const courseType&) const;
    bool operator<(const courseType&) const;
    bool operator>=(const courseType&) const;
    bool operator>(const courseType&) const;

    courseType(string cName = "", string cNo = "",
               char grade = '*', int credits = 0);
      //Constructor
      //The object is initialized according to the parameters.
      //Postcondition: courseName = cName; courseNo = cNo;
      //     courseGrade = grade; courseCredits = credits;

private:
    string courseName;  //variable to store the course name
    string courseNo;    //variable to store the course number
    char courseGrade;   //variable to store the grade
    int courseCredits;  //variable to store the course credits
};
```

Figure 4-2 shows the UML class diagram of the `class courseType`.

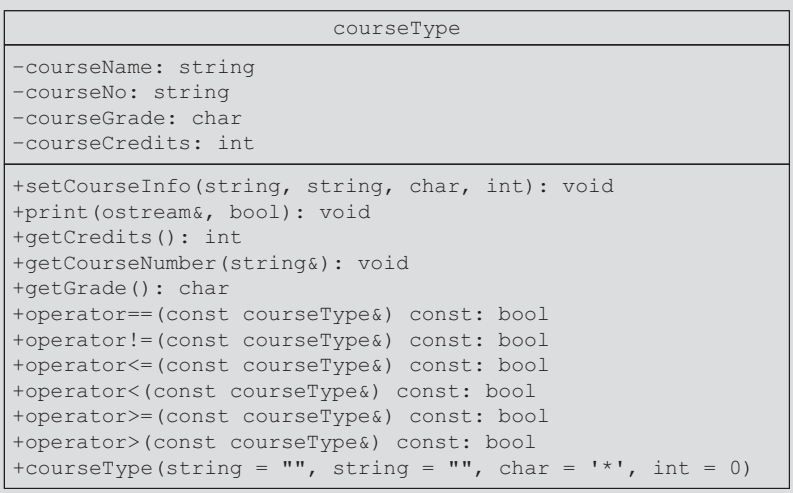


FIGURE 4-2 UML class diagram of the `class courseType`

Next, we discuss the definition of the functions to implement the operations of the `class courseType`. These definitions are quite straightforward and easy to follow.

The function `setCourseInfo` sets the values of the `private` data members according to the values of the parameters. Its definition is as follows:

```
void courseType::setCourseInfo(string cName, string cNo,
                               char grade, int credits)
{
    courseName = cName;
    courseNo = cNo;
    courseGrade = grade;
    courseCredits = credits;
}
```

The function `print` prints the course information. If the `bool` parameter `isGrade` is `true`, the grade is printed on the screen; otherwise, three stars are shown in place of the grade. Also, we print the course name and course number left-justified rather than right-justified (the default). Thus, we need to set the `left` manipulator. This manipulator will be unset before we print the grade and the credit hours. The following steps describe this function:

1. Set the `left` manipulator.
2. Print the course number.
3. Print the course name.
4. Unset the `left` manipulator.
5. Print the credit hours.
6. If `isGrade` is `true`

   Output the grade
   else
   Output three stars.

The definition of the function `print` is as follows:

```
void courseType::print(ostream& outp, bool isGrade)
{
    outp << left;                                   //Step 1
    outp << setw(8) << courseNo << "   ";           //Step 2
    outp << setw(15) << courseName;                 //Step 3
    outp.unsetf(ios::left);                         //Step 4
    outp << setw(3) << courseCredits << "   ";      //Step 5

    if (isGrade)                                     //Step 6
        outp << setw(4) << courseGrade << endl;
    else
        outp << setw(4) << "***" << endl;
}
```

The constructor is declared with default values. If no values are specified when a `courseType` object is declared, the constructor uses the default to initialize the object. Using the default values, the object's data members are initialized as follows: `courseNo` to blank, `courseName` to blank, `courseGrade` to `*`, and `creditHours` to `0`. Otherwise, the values specified in the object declaration are used to initialize the object. Its definition is as follows:

```
courseType::courseType(string cName, string cNo,
                       char grade, int credits)
{
    setCourseInfo(cName, cNo, grade, credits);
}
```

The definitions of the remaining functions are straightforward.

```
int courseType::getCredits()
{
    return courseCredits;
}

char courseType::getGrade()
{
    return courseGrade;
}

void courseType::getCourseNumber(string& cNo)
{
    cNo = courseNo;
}

bool courseType::operator==(const courseType& right) const
{
    return (courseNo == right.courseNo);
}


bool courseType::operator!=(const courseType& right) const
{
    return (courseNo != right.courseNo);
}

bool courseType::operator<=(const courseType& right) const
{
    return (courseNo <= right.courseNo);
}

bool courseType::operator<(const courseType& right) const
{
    return (courseNo < right.courseNo);
}

bool courseType::operator>=(const courseType& right) const
{
    return (courseNo >= right.courseNo);
}

bool courseType::operator>(const courseType& right) const
{
    return (courseNo > right.courseNo);
}
```

Next we discuss the component student.

Student

The main characteristics of a student are the student name, student ID, number of courses in which enrolled, courses in which enrolled, and the grade for each course. Because every student has to pay tuition, we also include a member to indicate whether the student has paid the tuition.

Every student is a person, and every student takes courses. We have already designed a `class personType` to process a person's first name and last name. We have also designed a class to process the information of a course. Thus, we see that we can derive the `class studentType` to keep track of a student's information from the `class personType`, and one member of this class is of type `courseType`. We can add more members as needed.

The basic operations to be performed on an object of type `studentType` are as follows:

1. Set the student information.
2. Print the student information.
3. Calculate the number of credit hours taken.
4. Calculate the GPA.
5. Calculate the billing amount.
6. Because the grade report will print the courses in ascending order, sort the courses according to the course number.

The following class defines `studentType` as an ADT. We assume that a student takes no more than six courses per semester:

```
class studentType: public personType
{
public:
    void setInfo(string fname, string lName, int ID,
                 bool isTPaid,
                 vector<courseType> courses);
      //Function to set the student's information
      //The private data members are set according
      //to the parameters.

    void print(ostream& out, double tuitionRate);
      //Function to print the student's grade report
      //The output is stored in a file specified by the
      //parameter out.

    studentType();
      //Default constructor
      //Postcondition: Data members are initialized to
       //the default values.
```

```
    int getHoursEnrolled();
      //Function to return the credit hours a student
      //is enrolled in.
      //Postcondition: The number of credit hours in which a
      //   student is enrolled is calculated and returned.

    double getGpa();
      //Function to return the grade point average.
      //Postcondition: The GPA is calculated and returned.

    double billingAmount(double tuitionRate);
      //Function to return the tuition fees
      //Postcondition: The tuition fees due is calculated
      //               and returned.

private:
    int sId;               //variable to store the student ID
    int numberOfCourses;   //variable to store the number
                           //of courses
    bool isTuitionPaid;    //variable to indicate if the tuition
                           //is paid
    vector<courseType> coursesEnrolled;//vector to store the courses
};
```

Figure 4-3 shows the UML class diagram of the **class studentType** and the inheritance hierarchy.

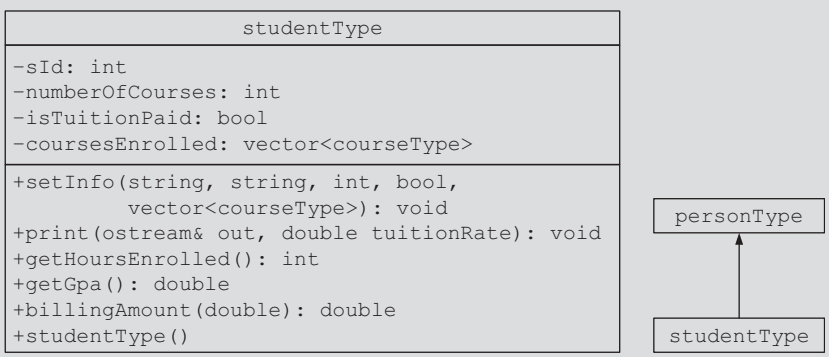


FIGURE 4-3 UML class diagram of the `class studentType` and the inheritance hierarchy

Next, we discuss the definitions of the functions to implement the operations of the **class studentType**.

The function **setInfo** first initializes the **private** data members according to the incoming parameters. The **class studentType** is derived from the **class personType**, and the variables to store the first name and last name are **private** data

members of that class. Therefore, we call the member function `setName` of the `class personType`, and we pass the appropriate variables to set the first and last names. To sort the array `coursesEnrolled` we use the algorithm `sort` provided by the STL.

To use the algorithm `sort`, to sort the vector `coursesEnrolled`, we need to know the position of the first element and last element in the vector `coursesEnrolled`. When we declare the vector `coursesEnrolled`, we did not specify its size. The function `begin` of the `class vector` returns the position of the first element in a `vector` container; the function `end` specifies the position of the last element. Therefore, `coursesEnrolled.begin()` specifies the position of the first element of the vector `coursesEnrolled`, and `coursesEnrolled.end()` specifies the position of the last element. Now the operator `<=` is overloaded for the `class courseType` and it compares the courses by the course number; the `sort` algorithm will use this criteria to sort the vector `coursesEnrolled`. The following statement sorts the vector `coursesEnrolled`.

```
sort(coursesEnrolled.begin(), coursesEnrolled.end());
```

The definition of the function `setInfo` is as follows:

```
void studentType::setInfo(string fName, string lName, int ID,
                          bool isTPaid,
                          vector<courseType> courses)
{
    setName(fName, lName);

    sId = ID;
    isTuitionPaid = isTPaid;
    numberOfCourses = courses.size();

    coursesEnrolled = courses;

    sort(coursesEnrolled.begin(), coursesEnrolled.end());
}
```

The default constructor initializes the `private` data members to the default values. Note that because the `private` data member `coursesEnrolled` is of type `vector`, the default constructor of the `class vector` executes automatically and initializes `coursesEnrolled`.

```
studentType::studentType()
{
    numberOfCourses = 0;
    sId = 0;
    isTuitionPaid = false;
}
```

The function `print` prints the grade report. If the student has paid his or her tuition, the grades and the GPA are shown. Otherwise, three stars are printed in place of each grade, the GPA is not shown, a message indicates that the grades are being held for nonpayment of the tuition, and the amount due is shown. This function has the following steps:

1. Output the student's name.
2. Output the student's ID.
3. Output the number of courses in which enrolled.
4. Output heading: `CourseNo CourseName Credits Grade`
5. Print each course's information.
6. Print the total credit hours.
7. To output the GPA and billing amount in a fixed decimal format with the decimal point and trailing zeros, set the necessary flag. Also, set the precision to two decimal places.
8. `if isTuitionPaid` is `true`
   Output the GPA
   `else`
   Output the billing amount and a message about withholding the grades.

This definition of the function `print` is as follows:

```
void studentType::print(ostream& outp, double tuitionRate)
{
    outp << "Student Name: " << personType::getFirstName()
         << " " << personType::getLastName() << endl;   //Step 1

    outp << "Student ID: " << sId << endl;              //Step 2

    outp << "Number of courses enrolled: "
         << numberOfCourses << endl << endl;            //Step 3

    outp << left;
    outp << "Course No" << setw(15) << "  Course Name"
         << setw(8) << "Credits"
         << setw(6) << "Grade" << endl;                 //Step 4

    outp.unsetf(ios::left);

    for (int i = 0; i < numberOfCourses; i++)
        coursesEnrolled[i].print(outp, isTuitionPaid);  //Step 5
    outp << endl;

    outp << "Total number of credit hours: "
         << getHoursEnrolled() << endl;                 //Step 6

    outp << fixed << showpoint << setprecision(2);      //Step 7
```

```
    if (isTuitionPaid)                                    //Step 8
        outp << "Midsemester GPA: " << getGpa() << endl;
    else
    {
        outp << "*** Grades are being held for not paying "
             << "the tuition. ***" << endl;
        outp << "Amount Due: $" << billingAmount(tuitionRate)
             << endl;
    }

    outp << "-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*"
         << "-*-*-*-*-" << endl << endl;
}
```

The function `getHoursEnrolled` calculates and returns the total credit hours that a student is taking. These credit hours are needed to calculate both the GPA and the billing amount. The total credit hours are calculated by adding the credit hours of each course in which the student is enrolled. The credit hours for a course are in the `private` data member of an object of type `courseType`. Therefore, we use the member function `getCredits` of the `class courseType` to retrieve the credit hours. The definition of this function is as follows:

```
int studentType::getHoursEnrolled()
{
    int totalCredits = 0;

    for (int i = 0; i < numberOfCourses; i++)
        totalCredits += coursesEnrolled[i].getCredits();
    return totalCredits;
}
```

If a student has not paid the tuition, the function `billingAmount` calculates and returns the amount due, based on the number of credit hours enrolled. The definition of this function is as follows:

```
double studentType::billingAmount(double tuitionRate)
{
    return tuitionRate * getHoursEnrolled();
}
```

We now discuss the function `getGpa`. This function calculates a student's GPA. To find the GPA, we find the equivalent points for each grade, add the points, and then divide the sum by the total credit hours the student is taking. The definition of this function is as follows:

```
double studentType::getGpa()
{
    double sum = 0.0;
```

```
    for (int i = 0; i < numberOfCourses; i++)
    {
        switch (coursesEnrolled[i].getGrade())
        {
        case 'A':
            sum += coursesEnrolled[i].getCredits() * 4;
            break;

        case 'B':
            sum += coursesEnrolled[i].getCredits() * 3;
            break;

        case 'C':
            sum += coursesEnrolled[i].getCredits() * 2;
            break;

        case 'D':
            sum += coursesEnrolled[i].getCredits() * 1;
            break;

        case 'F':
            break;

        default:
            cout << "Invalid Course Grade" << endl;
        }
    }

    if (getHoursEnrolled() != 0)
        return sum / getHoursEnrolled();
    else
        return 0;
}
```

MAIN PROGRAM

Now that we have designed the `class`es `courseType` and `studentType`, we will use these classes to complete the program.

Because the function `print` of the class does the necessary computations to print the final grade report, the main program has very little work to do. In fact, all that is left for the main program is to declare the objects to hold the students' data, load the data into these objects, and then print the grade reports. Because the input is in a file and the output will be sent to a file, we declare stream variables to access the input and output files. Essentially, the main algorithm for the program is as follows:

1. Declare the variables.
2. Open the input file.
3. If the input file does not exist, exit the program.
4. Open the output file.

5. Get the tuition rate.
6. Load the students' data.
7. Print the grade reports.

Variables

To store students' data, we use the vector container, `studentList`, whose elements are of type `studentType`. We also need to store the tuition rate. Because the data will be read from a file, and because the output is sent to a file, we need two stream variables to access the input and output files. Thus, we need the following variables:

```
vector<studentType> studentList; //vector to store the
                                 // students' data

double tuitionRate;    //variable to store the tuition rate

ifstream infile;    //input stream variable
ofstream outfile;   //output stream variable
```

To simplify the complexity of the function `main`, we write a function, `getStudentData`, to load the students' data and another function, `printGradeReports`, to print the grade reports. The next two sections describe these functions.

Function getStudentData

This function has two parameters: a parameter to access the input file and a parameter to access the vector container `studentList`. In pseudocode, the definition of this function is as follows:

For each student in the university,

1. Get the first name, last name, student ID, and `isPaid`.
2. `if isPaid` is `'Y'`
   set `isTuitionPaid` to `true`
   `else`
   set `isTuitionPaid` to `false`
3. Get the number of courses the student is taking.
4. For each course
   a. Get the course name, course number, credit hours, and grade.
   b. Load the course information into a `courseType` object.
   c. Push the object containing course information into the vector container that stores course data.
5. Load the data into a `studentType` object.
6. Push the object containing student's data into `studentList`.

We need to declare several local variables to read and store the data. The definition of the function `getStudentData` is as follows:

```
void getStudentData(ifstream& infile,
                    vector<studentType> &studentList)
{
     //Local variable
    string fName;     //variable to store the first name
    string lName;     //variable to store the last name
    int ID;           //variable to store the student ID
    int noOfCourses;  //variable to store the number of courses
    char isPaid;      //variable to store Y/N, that is,
                      //is tuition paid
    bool isTuitionPaid; //variable to store true/false

    string cName;  //variable to store the course name
    string cNo;    //variable to store the course number
    int credits;   //variable to store the course credit hours
    char grade;    //variable to store the course grade

    vector<courseType> courses; //vector of objects to store course
                                //information
    courseType cTemp;
    studentType sTemp;

    infile >> fName;                                    //Step 1

    while (infile)
    {
        infile >> lName >> ID >> isPaid;                //Step 1

        if (isPaid == 'Y')                              //Step 2
            isTuitionPaid = true;
        else
            isTuitionPaid = false;

        infile >> noOfCourses;                          //Step 3

        courses.clear();

        for (int i = 0; i < noOfCourses; i++)           //Step 4
        {
            infile >> cName >> cNo >> credits >> grade; //Step 4.a
            cTemp.setCourseInfo(cName, cNo,
                                grade, credits);        //Step 4.b
            courses.push_back(cTemp);                   //Step 4.c
        }

        sTemp.setInfo(fName, lName, ID, isTuitionPaid,
                      courses);                         //Step 5
        studentList.push_back(sTemp);                   //Step 6

        infile >> fName;                                //Step 1
    }//end while
}
```

Function printGradeReports

This function prints the grade reports. For each student, it calls the function `print` of the `class studentType` to print the grade report. The definition of the function `printGradeReports` is as follows:

```
void printGradeReports(ofstream& outfile,
                       vector<studentType> studentList,
                       double tuitionRate)
{
    for (int count = 0; count < studentList.size(); count++)
        studentList[count].print(outfile, tuitionRate);
}
```

MAIN PROGRAM

```
//*****************************************************************
// Author: D.S. Malik
//
// This program illustrates how to use the classes courseType,
// studentType, and vector.
//*****************************************************************

#include <iostream>
#include <fstream>
#include <string>
#include <algorithm>
#include <vector>
#include <iterator>

#include "studentType.h"

using namespace std;

void getStudentData(ifstream& infile,
                    vector<studentType> &studentList);

void printGradeReports(ofstream& outfile,
                       vector<studentType> studentList,
                       double tuitionRate);

int main()
{
    vector<studentType> studentList;

    double tuitionRate;

    ifstream infile;
    ofstream outfile;

    infile.open("stData.txt");
```

```
    if (!infile)
    {
        cout << "Input file does not exist. "
             << "Program terminates." << endl;
        return 1;
    }

    outfile.open("stDataOut.txt");

    infile >> tuitionRate;    //get the tuition rate

    getStudentData(infile, studentList);
    printGradeReports(outfile, studentList, tuitionRate);

    return 0;
}

//Place the definition of the function getStudentData here
//Place the definition of the function printGradeReports here
```

Sample Run:

```
Student Name: Lisa Miller
Student ID: 890238
Number of courses enrolled: 4

Course No  Course Name  Credits Grade
CSC478     ComputerSci      3      B
HIS356     History          3      A
MTH345     Mathematics      4      A
PHY357     Physics          3      B

Total number of credit hours: 13
Midsemester GPA: 3.54
-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-

Student Name: Bill Wilton
Student ID: 798324
Number of courses enrolled: 5

Course No  Course Name  Credits Grade
BIO234     Biology          4    ***
CHM256     Chemistry        4    ***
ENG378     English          3    ***
MTH346     Mathematics      3    ***
PHL534     Philosophy       3    ***

Total number of credit hours: 17
*** Grades are being held for not paying the tuition. ***
Amount Due: $5865.00
-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-
```

```
Student Name: Dandy Goat
Student ID: 746333
Number of courses enrolled: 6

Course No  Course Name  Credits Grade
BUS128     Business         3      C
CHM348     Chemistry        4      B
CSC201     ComputerSci      3      B
ENG328     English          3      B
HIS101     History          3      A
MTH137     Mathematics      3      A

Total number of credit hours: 19
Midsemester GPA: 3.16
-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-
```

Input File

```
345
Lisa Miller 890238 Y 4
Mathematics MTH345 4 A
Physics PHY357 3 B
ComputerSci CSC478 3 B
History HIS356 3 A

Bill Wilton 798324 N 5
English ENG378 3 B
Philosophy PHL534 3 A
Chemistry CHM256 4 C
Biology BIO234 4 A
Mathematics MTH346 3 C

Dandy Goat 746333 Y 6
History HIS101 3 A
English ENG328 3 B
Mathematics MTH137 3 A
Chemistry CHM348 4 B
ComputerSci CSC201 3 B
Business BUS128 3 C
```

QUICK REVIEW

1. The STL provides class templates that process lists, stacks, and queues.
2. The three main components of the STL are containers, iterators, and algorithms.
3. STL containers are class templates.
4. Iterators are used to step through the elements of a container.

5. Algorithms are used to manipulate the elements in a container.
6. The main categories of containers are sequence containers, associative containers, and container adapters.
7. The three predefined sequence containers are `vector`, `deque`, and `list`.
8. A vector container stores and manages its objects in a dynamic array.
9. Because an array is a random access data structure, elements of a vector can be accessed randomly.
10. The name of the class that implements the vector container is `vector`.
11. Item insertion in a vector container is accomplished by using the operations `insert` and `push_back`.
12. Item deletion in a vector container is accomplished by using the operations `pop_back`, `erase`, and `clear`.
13. An iterator to a vector container is declared using the `typedef iterator`, which is declared as a `public` member of the `class vector`.
14. Member functions common to all containers are the default constructor, constructors with parameters, the copy constructor, the destructor, `empty`, `size`, `max_size`, `swap`, `begin`, `end`, `rbegin`, `rend`, `insert`, `erase`, `clear`, and the relational operator functions.
15. The member function `begin` returns an iterator to the first element into the container.
16. The member function `end` returns an iterator to the last element into the container.
17. In addition to the member functions listed in 14, the other member functions common to all sequence containers are `insert`, `push_back`, `pop_back`, `erase`, `clear`, and `resize`.
18. The `copy` algorithm is used to copy the elements in a given range to another place.
19. The function `copy`, using an `ostream` iterator, can also be used to output the elements of a container.
20. When we create an iterator of the type `ostream`, we also specify the type of element that the iterator will output.
21. Deque containers are implemented as dynamic arrays in such a way that the elements can be inserted at both ends of the array.
22. A `deque` can expand in either direction.
23. The name of the header file containing the definition of the `class deque` is `deque`.
24. In addition to the operations that are common to all containers, the other operations that can be used to manipulate the elements of a `deque` are `assign`, `push_front`, `pop_front`, `at`, array subscripting operator `[]`, `front`, and `back`.

25. The five categories of iterators are: input, output, forward, bidirectional, and random access iterator.
26. Input iterators are used to input data from an input stream.
27. Output iterators are used to output data to an output stream.
28. A forward iterator can refer to the same element in the same collection and process the same element more than once.
29. Bidirectional iterators are forward iterators that can also iterate backwards over the elements.
30. Bidirectional iterators can be used with containers of type `list`, `set`, `multiset`, `map`, and `multimap`.
31. Random access iterators are bidirectional iterators that can randomly process the elements of a container.
32. Random access iterators can be used with containers of type `vector`, `deque`, `string`, and arrays.

EXERCISES

1. What are the three main components of the STL?
2. What is the difference between an STL container and an STL iterator?
3. Write a statement that declares a vector object that can store 50 decimal numbers.
4. Write a statement that declares and stores the elements of the following array into a `vector` object:

```
char vowels[5] = {'a', 'e', 'i', 'o', 'u'};
```

5. Write a statement to declare `screen` to be an `ostream_iterator` initialized to the standard output device that outputs the elements of an `int` vector object.
6. Consider the following statements:

```
vector<int> intVector;
```

Suppose that `intVector = {5, 7, 9, 11, 13}`. Moreover, suppose that `screen` is an `ostream_iterator` initialized to the standard output device to output the elements of an `int` vector object. What is the effect of the following statement?

```
copy(vecList.begin(), vecList.end(), screen);
```

7. What is the output of the following program segment?

```
vector<int> vecList(5);

for (int j = 0; j < 5; j++)
    vecList[j] = 2 * j;
for (int j = 0; j < 5; j++)
    cout << vecList[j] << " ";
cout << endl;
```

8. What is the output of the following program segment? (Assume that `screen` is an `ostream_iterator` initialized to the standard output device to output elements of type `int`.)

```
int list[5] = {2,4,6,8,10};
vector<int> vecList(5);

copy(list, list + 5, vecList.begin());

copy(vecList.begin(), vecList.end(), screen);
cout << endl;
```

9. What is the output of the following program segment? (Assume that `screen` is an `ostream_iterator` initialized to the standard output device to output elements of type `int`.)

```
vector<int> vecList;
vector<int>::iterator vecIt;

vecList.push_back(3);
vecList.push_back(5);
vecList.push_back(7);
vecIt = vecList.begin();
++vecIt;
vecList.erase(vecIt);
vecList.push_back(9);

copy(vecList.begin(), vecList.end(), screen);
cout << endl;
```

10. What is the output of the following program segment? (Assume that `screen` is an `ostream_iterator` initialized to the standard output device to output elements of type `int`.)

```
int list[5] = {2,4,6,8,10};
vector<int> vecList(7);

copy(list, list + 5, vecList.begin());

vecList.push_back(12);

copy(vecList.begin(), vecList.end(), screen);
cout << endl;
```

11. What is the output of the following program segment? (Assume that `screen` is an `ostream_iterator` initialized to the standard output device to output elements of type `double`.)

```
vector<double> sales(3);

sales[0] = 50.00;
sales[1] = 75.00;
sales[2] = 100.00;
```

```
sales.resize(5);

sales[3] = 200.00;
sales[4] = 95.00;

copy(sales.begin(), sales.end(), screen);
cout << endl;
```

12. What is the output of the following program segment? (Assume that `screen` is an `ostream_iterator` initialized to the standard output device that outputs elements of type `int`.)

```
vector<int> intVector;
vector<int>::iterator vecIt;

intVector.push_back(15);
intVector.push_back(2);
intVector.push_back(10);
intVector.push_back(7);
vecIt = intVector.begin();
vecIt++;
intVector.erase(vecIt);
intVector.pop_back();

copy(intVector.begin(),intVector.end(), screen);
```

13. Suppose that `vecList` is a `vector` container and

```
vecList = {12, 16, 8, 23, 40, 6, 18, 9, 75}
```

Show `vecList` after the following statement executes:

```
copy(vecList.begin() + 2, vecList.end(), vecList.begin());
```

14. Suppose that `vecList` is a `vector` container and

```
vecList = {12, 16, 8, 23, 40, 6, 18, 9, 75}
```

Show `vecList` after the following statement executes:

```
copy(vecList.rbegin() + 3, vecList.rend(), vecList.rbegin());
```

15. What is the output of the following program segment?

```
deque<int>  intDeq;
ostream_iterator<int> screen(cout, " ");
deque<int>::iterator deqIt;

intDeq.push_back(5);
intDeq.push_front(23);
intDeq.push_front(45);
intDeq.push_back(35);
intDeq.push_front(0);
intDeq.push_back(50);
intDeq.push_front(34);

deqIt = intDeq.begin();
intDeq.insert(deqIt,76);
intDeq.pop_back();
```

```
deqIt = intDeq.begin();
++deqIt;
++deqIt;

intDeq.erase(deqIt);
intDeq.push_front(2 * intDeq.back());
intDeq.push_back(3 * intDeq.front());

copy(intDeq.begin(), intDeq.end(), screen);
cout << endl;
```


PROGRAMMING EXERCISES

1. Write a program that allows the user to enter the last names of five candidates in a local election and the votes received by each candidate. The program should then output each candidate's name, votes received by that candidate, and the percentage of the total votes received by the candidate. Your program should also output the winner of the election. A sample output is as follows:

```
Candidate     Votes Received     % of Total Votes
Johnson         5000               25.91
Miller          4000               20.72
Duffy           6000               31.09
Robinson        2500               12.95
Sam             1800                9.33
Total           19300
The Winner of the Election is Duffy.
```

2. Write a program that allows the user to input the students' names followed by their test scores and outputs the following:
   a. Class average
   b. Names of all students whose test scores are below the class average with an appropriate message
   c. Highest test score and the names of all students having the highest score
3. Write a program that uses a vector object to store a set of real numbers. The program outputs the smallest, largest, and average of the numbers. When declaring the vector object, do not specify its size. Use the function `push_back` to insert elements in the vector object.
4. Write the definition of the function template `reverseVector` to reverse the elements of a vector object.

```
template<class elemType>
void reverseVector(vector<elemType> &list);
  //Reverses the elements of the vector list.
  //Example: Suppose list = {4, 8, 2, 5}.
  //         After a call to this function, list = {5, 2, 8, 4}.
```

Also, write a program to test the function `reverseVector`. When declaring the vector object, do not specify its size. Use the function `push_back` to insert elements in the vector object.

5. Write the definition of the function template `seqSearch` to implement the sequential search on a vector object.

```
template<class elemType>
int seqSearch(const vector<elemType> &list, const elemType& item);
  //If item is found in the list, returns the
  //position of the item in the list; otherwise, returns -1.
```

Also, write a program to test the function `seqSearch`. Use the function `push_back` to insert elements in the vector object.

6. Write a program to find the mean and standard deviation of numbers. The mean (average) of n numbers $x_1, x_2, \ldots, x_n$ is $x = (x_1 + x_2 + \ldots + x_n) / n$. The standard deviation of these numbers is as follows:

$$s = \sqrt{\frac{(x_1 - x)^2 + (x_2 - x)^2 + \cdots + (x_i - x)^2 + \cdots + (x_n - x)^2}{n}}$$

Use a vector object to store the numbers.

7. a. Some of the characteristics of a book are the title, author(s), publisher, ISBN, price, and year of publication. Design the `class bookType` that defines the book as an ADT.

   Each object of the `class bookType` can hold the following information about a book: title, up to four authors, publisher, ISBN, price, and number of copies in stock. To keep track of the number of authors, add another data member.

   Include the member functions to perform the various operations on the objects of `bookType`. For example, the typical operations that can be performed on the title are to show the title, set the title, and check whether a title is the same as the actual title of the book. Similarly, the typical operations that can be performed on the number of copies in stock are to show the number of copies in stock, set the number of copies in stock, update the number of copies in stock, and return the number of copies in stock. Add similar operations for the publisher, ISBN, book price, and authors. Add the appropriate constructors and a destructor (if one is needed).

   b. Write the definitions of the member functions of the `class bookType`.

   c. Write a program that uses the `class bookType` and tests the various operations on the objects of `class bookType`. Declare a vector container of type `bookType`. Some of the operations that you should perform are to search for a book by its title, search by ISBN, and update the number of copies in stock.

8. a. In the first part of this exercise, you will design a `class memberType`.

   i. Each object of `memberType` can hold the name of a person, member ID, number of books bought, and amount spent.

   ii. Include the member functions to perform the various operations on the objects of `memberType`—for example, modify, set, and show a person's name. Similarly, update, modify, and show the number of books bought and the amount spent.

   iii. Add the appropriate constructors and a destructor (if one is needed).

   iv. Write the definitions of the member functions of `memberType`.

   b. Using the classes designed in Programming Exercise 7 and part (8a), write a program to simulate a bookstore. The bookstore has two types of customers: those who are members of the bookstore and those who buy books from the bookstore only occasionally. Each member has to pay a $10 yearly membership fee and receives a 5% discount on each book bought.

   For each member, the bookstore keeps track of the number of books bought and the total amount spent. For every eleventh book that a member buys, the bookstore takes the average of the total amount of the last 10 books bought, applies this amount as a discount, and then resets the total amount spent to 0.

   Your program should contain a menu that gives the user different choices to effectively run the program; in other words, your program should be self-driven.

9. Redo Programming Exercise 9 of Chapter 3 so that the address book is stored in a `vector` object.

10. (**Stock Market**) Write a program to help a local stock trading company automate its systems. The company invests only in the stock market. At the end of each trading day, the company would like to generate and post the listing of its stocks so that investors can see how their holdings performed that day. We assume that the company invests in, say, 10 different stocks. The desired output is to produce two listings, one sorted by stock symbol and another sorted by percent gain from highest to lowest.
The input data is stored in a file in the following format:

```
symbol openingPrice closingPrice todayHigh todayLow prevClose
volume
```

For example, the sample data is as follows:

```
MSMT 112.50 115.75 116.50 111.75 113.50 6723823
CBA 67.50 75.50 78.75 67.50 65.75 378233
.
.
.
```

The first line indicates that the stock symbol is `MSMT`, today's opening price was `112.50`, the closing price was `115.75`, today's high price was `116.50`, today's low price was `111.75`, yesterday's closing price was `113.50`, and the number of shares currently being held is `6723823`.

The listing sorted by stock symbols must be in the following form:

```
*********  First Investor's Heaven  **********
*********     Financial Report       **********
Stock        Today              Previous  Percent
Symbol Open   Close  High    Low  Close  Gain Volume
------ ----   -----  ----    ---  -----  ---- ------
ABC   123.45 130.95 132.00 125.00 120.50  8.67% 10000
AOLK   80.00  75.00  82.00  74.00  83.00 -9.64%  5000
CSCO  100.00 102.00 105.00  98.00 101.00  0.99% 25000
IBD    68.00  71.00  72.00  67.00  75.00 -5.33% 15000
MSET  120.00 140.00 145.00 140.00 115.00 21.74% 30920
Closing Assets: $9628300.00
-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*-*
```

Develop this programming exercise in two steps. In the first step (part a), design and implement a stock object. In the second step (part b), design and implement an object to maintain a list of stocks.

a. (**Stock Object**) Design and implement the stock object. Call the class that captures the various characteristics of a stock object `stockType`.

   The main components of a stock are the stock symbol, stock price, and number of shares. Moreover, we need to output the opening price, high price, low price, previous price, and the percent gain/loss for the day. These are also all the characteristics of a stock. Therefore, the stock object should store all this information.

   Perform the following operations on each stock object:

   i. Set the stock information.
   ii. Print the stock information.
   iii. Show the different prices.
   iv. Calculate and print the percent gain/loss.
   v. Show the number of shares.

   a.1. The natural ordering of the stock list is by stock symbol. Overload the relational operators to compare two stock objects by their symbols.

   a.2. Overload the insertion operator, <<, for easy output.

   a.3. Because data is stored in a file, overload the stream extraction operator, >>, for easy input.
   For example, suppose `infile` is an `ifstream` object and the input file was opened using the object `infile`. Further suppose that `myStock` is a stock object. Then, the statement

   ```
   infile >> myStock;
   ```

reads data from the input file and stores it in the object `myStock`. (Note that this statement reads and stores data in relevant components of `myStock`.)

b. Now that you have designed and implemented the `class stockType` to implement a stock object in a program, it is time to create a list of stock objects. Let us call the class to implement a list of stock objects **`stockListType`**. To store the list of stocks, you need to declare a `vector`. The component type of this `vector` is `stockType`.

Because the company also requires you to produce the list ordered by the percent gain/loss, you need to sort the stock list by this component. However, you are not to physically sort the list by the component percent gain/loss; instead, you will provide a logical ordering with respect to this component.

To do so, add a data member, a vector, to hold the indices of the stock list ordered by the component percent gain/loss. Call this array `indexByGain`. When printing the list ordered by the component percent gain/loss, use the array `indexByGain` to print the list. The elements of the array `indexByGain` will tell which component of the stock list to print next. In skeleton form, the definition of the `class stockListType` is as follows:

```
class stockListType
{
public:
    void insert(const stockType& item));
       //Function to insert a stock in the list.
...

private:
   vector<int> indexByGain;
   vector<stockType> list; //vector to store the list //of stocks
};
```

c. Write a program that uses these two classes to automate the company's analysis of stock data.

This page intentionally left blank